

Overview of Data Frame Operations

Data frames are the workhorse of R, so in this lecture we will basically be creating a "cheat sheet" of common operations used with data frames and R. This will be a very useful lecture, and while we will be going over material we've already covered its vital to know data frames very well in order to save yourself tons of time in the future!

We're going to do an overview of the following common actions:

- Creating Data Frames
- Importing and Exporting Data
- Getting Information about Data Frame
- Referencing Cells
- Referencing Rows
- Referencing Columns
- Adding Rows
- Adding Columns
- Setting Column Names
- Selecting Multiple Rows
- Selecting Multiple Columns
- Dealing with Missing Data

Creating Data Frames

In [79]:

```
empty <- data.frame() # empty data frame

c1 <- 1:10 # vector of integers

c2 <- letters[1:10] # vector of strings

df <- data.frame(col.name.1=c1,col.name.2=c2)
```

In [59]:

```
df
```

Out[59]:

	col_name_1	col_name_2
1	1	a
2	2	b
3	3	c
4	4	d
5	5	e
6	6	f
7	7	g
8	8	h
9	9	i
10	10	j

Importing and Exporting Data

In []:

```
d2 <- read.csv('some.file.name.csv')

# For Excel Files
# Load the readxl package
library(readxl)
# Call info from the sheets using read.excel
df <- read_excel('Sample-Sales-Data.xlsx', sheet='Sheet1')

# Output to csv
write.csv(df, file='some.file.csv')
```

Getting Information about Data Frame

In [6]:

```
# Row and columns counts
nrow(df)
ncol(df)
```

Out[6]:

10

Out[6]:

2

In [7]:

```
# Column Names
colnames(df)
```

Out[7]:

'col_name_1' 'col_name_2'

In [9]:

```
# Row names (may just return index)
rownames(df)
```

Out[9]:

'1' '2' '3' '4' '5' '6' '7' '8' '9' '10'

Referencing Cells

You can think of the basics as using two sets of brackets for a single cell, and a single set of brackets for multiple cells. For example:

In [32]:

```
vec <- df[[5, 2]] # get cell by [[row,col]] num

newdf <- df[1:5, 1:2] # get multiplt cells in new df

df[[2, 'col.name.1']] <- 99999 # reassign a single cell
```

In [33]:

df

Out[33]:

	col_name_1	col_name_2
1	1	a
2	99999	b
3	3	c
4	4	d
5	5	e
6	6	f
7	7	g
8	8	h
9	9	i
10	10	j

Referencing Rows

Usually you'll use the [row,] format

In [43]:

```
# returns a data frame (and not a vector!)
rowdf <- df[1, ]
```

In [44]:

```
rowdf
```

Out[44]:

	col_name_1	col_name_2
1	1	a

In [49]:

```
# to get a row as a vector, use following
vrow <- as.numeric(as.vector(df[1,]))
```

In [50]:

```
vrow
```

Out[50]:

```
1 1
```

Referencing Columns

Most column references return a vector:

In [62]:

```
cars <- mtcars
head(cars)
```

Out[62]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21	6	160	110	3.9	2.62	16.46	0	1	4	4
Mazda RX4 Wag	21	6	160	110	3.9	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.32	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.44	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.46	20.22	1	0	3	1

In [63]:

```
colv1 <- cars$mpg # returns a vector
colv1

colv2 <- cars[, 'mpg'] # returns vector
colv2

colv3<- cars[, 1] # a is int or string
colv3

colv4 <- cars[['mpg']] # returns a vector
colv4
```

Out[63]:

```
21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4 14.7 32.4 30.4
33.9 21.5 15.5 15.2 13.3 19.2 27.3 26 30.4 15.8 19.7 15 21.4
```

Out[63]:

```
21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4 14.7 32.4 30.4
33.9 21.5 15.5 15.2 13.3 19.2 27.3 26 30.4 15.8 19.7 15 21.4
```

Out[63]:

```
21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4 14.7 32.4 30.4
33.9 21.5 15.5 15.2 13.3 19.2 27.3 26 30.4 15.8 19.7 15 21.4
```

Out[63]:

```
21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4 14.7 32.4 30.4
33.9 21.5 15.5 15.2 13.3 19.2 27.3 26 30.4 15.8 19.7 15 21.4
```

In [64]:

```
# Ways of Returning Data Frames
mpgdf <- cars['mpg'] # returns 1 col df
head(mpgdf)

mpgdf2 <- cars[1] # returns 1 col df
head(mpgdf2)
```

Out[64]:

	mpg
Mazda RX4	21
Mazda RX4 Wag	21
Datsun 710	22.8

Hornet 4 Drive	21.4
Hornet Sportabout	18.7
Valiant	18.1

Out[64]:

	mpg
Mazda RX4	21
Mazda RX4 Wag	21
Datsun 710	22.8
Hornet 4 Drive	21.4
Hornet Sportabout	18.7
Valiant	18.1

Adding Rows

In [75]:

```
# Both arguments are DFs)
df2 <- data.frame(col.name.1=2000,col.name.2='new' )
df2

# use rbind to bind a new row!
dfnew <- rbind(df,df2)
```

Out[75]:

	col_name_1	col_name_2
1	2000	new

In [76]:

```
dfnew
```

Out[76]:

	col_name_1	col_name_2
1	1	a
2	2	b
3	3	c
4	4	d
5	5	e
6	6	f
7	7	g
8	8	h
9	9	i
10	10	j
11	2000	new

Adding Columns

Adding Columns

In [80]:

```
df$newcol <- rep(NA, nrow(df)) # NA column
df
```

Out[80]:

	col_name_1	col_name_2	newcol
1	1	a	NA
2	2	b	NA
3	3	c	NA
4	4	d	NA
5	5	e	NA
6	6	f	NA
7	7	g	NA
8	8	h	NA
9	9	i	NA
10	10	j	NA

In [82]:

```
df[, 'copy.of.col2'] <- df$col.name.2 # copy a col
df
```

Out[82]:

	col_name_1	col_name_2	newcol	copy_of_col2
1	1	a	NA	a
2	2	b	NA	b
3	3	c	NA	c
4	4	d	NA	d
5	5	e	NA	e
6	6	f	NA	f
7	7	g	NA	g
8	8	h	NA	h
9	9	i	NA	i
10	10	j	NA	j

In [83]:

```
# Can also use equations!
df[['col1.times.2']] <- df$col.name.1 * 2
df
```

Out[83]:

	col_name_1	col_name_2	newcol	copy_of_col2	col1_times_2
1	1	a	NA	a	2
2	2	b	NA	b	4
3	3	c	NA	c	6
4	4	d	NA	d	8

5	col_name_1	col_name_2	newcol	copy_of_col2	col1_times_2
6	6	f	NA	f	12
7	7	g	NA	g	14
8	8	h	NA	h	16
9	9	i	NA	i	18
10	10	j	NA	j	20

In [84]:

```
df3 <- cbind(df, df$col.name.1)
df3
```

Out[84]:

	col_name_1	col_name_2	newcol	copy_of_col2	col1_times_2	df\$col_name_1
1	1	a	NA	a	2	1
2	2	b	NA	b	4	2
3	3	c	NA	c	6	3
4	4	d	NA	d	8	4
5	5	e	NA	e	10	5
6	6	f	NA	f	12	6
7	7	g	NA	g	14	7
8	8	h	NA	h	16	8
9	9	i	NA	i	18	9
10	10	j	NA	j	20	10

Setting Column Names

In [86]:

```
# Rename second column
colnames(df)[2] <- 'SECOND COLUMN NEW NAME'
df

# Rename all at once with a vector
colnames(df) <- c('col.name.1', 'col.name.2', 'newcol', 'copy.of.col2' , 'col1.times.2')
df
```

Out[86]:

	col_name_1	SECOND COLUMN NEW NAME	newcol	copy_of_col2	col1_times_2
1	1	a	NA	a	2
2	2	b	NA	b	4
3	3	c	NA	c	6
4	4	d	NA	d	8
5	5	e	NA	e	10
6	6	f	NA	f	12
7	7	g	NA	g	14
8	8	h	NA	h	16
9	9	i	NA	i	18
10	10	i	NA	i	20

	col_name_1	SECOND COLUMN NEW NAME	newcol	copy_of_col2	col1_times_2
--	------------	------------------------	--------	--------------	--------------

Out[86]:

	col_name_1	col_name_2	newcol	copy_of_col2	col1_times_2
1	1	a	NA	a	2
2	2	b	NA	b	4
3	3	c	NA	c	6
4	4	d	NA	d	8
5	5	e	NA	e	10
6	6	f	NA	f	12
7	7	g	NA	g	14
8	8	h	NA	h	16
9	9	i	NA	i	18
10	10	j	NA	j	20

Selecting Multiple Rows

In [87]:

```
first.ten.rows <- df[1:10, ] # Same as head(df, 10)
first.ten.rows
```

Out[87]:

	col_name_1	col_name_2	newcol	copy_of_col2	col1_times_2
1	1	a	NA	a	2
2	2	b	NA	b	4
3	3	c	NA	c	6
4	4	d	NA	d	8
5	5	e	NA	e	10
6	6	f	NA	f	12
7	7	g	NA	g	14
8	8	h	NA	h	16
9	9	i	NA	i	18
10	10	j	NA	j	20

In [88]:

```
everything.but.row.two <- df[-2, ]
everything.but.row.two
```

Out[88]:

	col_name_1	col_name_2	newcol	copy_of_col2	col1_times_2
1	1	a	NA	a	2
3	3	c	NA	c	6
4	4	d	NA	d	8
5	5	e	NA	e	10
6	6	f	NA	f	12

7	col_name_1	col_name_2	newcol	copy_of_col2	col1_times_2
8	8	h	NA	h	16
9	9	i	NA	i	18
10	10	j	NA	j	20

In [91]:

```
# Conditional Selection
sub1 <- df[ (df$col.name.1 > 8 & df$col1.times.2 > 10), ]
sub1

sub2 <- subset(df, col.name.1 > 8 & col1.times.2 > 10)
sub2
```

Out[91]:

	col_name_1	col_name_2	newcol	copy_of_col2	col1_times_2
9	9	i	NA	i	18
10	10	j	NA	j	20

Out[91]:

	col_name_1	col_name_2	newcol	copy_of_col2	col1_times_2
9	9	i	NA	i	18
10	10	j	NA	j	20

Selecting Multiple Columns

In [92]:

```
df[, c(1, 2, 3)] #Grab cols 1 2 3
```

Out[92]:

	col_name_1	col_name_2	newcol
1	1	a	NA
2	2	b	NA
3	3	c	NA
4	4	d	NA
5	5	e	NA
6	6	f	NA
7	7	g	NA
8	8	h	NA
9	9	i	NA
10	10	j	NA

In [93]:

```
df[, c('col.name.1', 'col1.times.2')] # by name
```

Out[93]:

	col_name_1	col1_times_2
--	------------	--------------

	col_name_1	col1_times_2
	col_name_1	col1_times_2
1	1	2
2	2	4
3	3	6
4	4	8
5	5	10
6	6	12
7	7	14
8	8	16
9	9	18
10	10	20

In [94]:

```
df[, -1] # keep all but first column
```

Out[94]:

	col_name_2	newcol	copy_of_col2	col1_times_2
1	a	NA	a	2
2	b	NA	b	4
3	c	NA	c	6
4	d	NA	d	8
5	e	NA	e	10
6	f	NA	f	12
7	g	NA	g	14
8	h	NA	h	16
9	i	NA	i	18
10	j	NA	j	20

In [95]:

```
df[, -c(1, 3)] # drop cols 1 and 3
```

Out[95]:

	col_name_2	copy_of_col2	col1_times_2
1	a	a	2
2	b	b	4
3	c	c	6
4	d	d	8
5	e	e	10
6	f	f	12
7	g	g	14
8	h	h	16
9	i	i	18
10	j	j	20

Dealing with Missing Data

Dealing with missing data is a very important skill to know when working with data frames!

In [97]:

```
any(is.na(df)) # detect anywhere in df
```

Out[97]:

FALSE

In [98]:

```
any(is.na(df$col.name.1)) # anywhere in col
```

Out[98]:

FALSE

In [101]:

```
# delete selected missing data rows  
df <- df[!is.na(df$col), ]
```

In [102]:

```
# replace NAs with something else  
df[is.na(df)] <- 0 # works on whole df
```

In [103]:

```
df$col[is.na(df$col)] <- 999 # For a selected column
```

All done!

Hope this helps! Remember to use this as a reference during your exercises and projects!